

# Hotel Booking System

## Teammate Setup Report

*How to reproduce the project environment, database, backend, and frontend*

**Purpose.** This document explains how a teammate can clone the GitHub repository, recreate the SQL Server database, load the image links needed for the homepage, start the backend, start the frontend, and verify that the project works.

**Current stack.** SQL Server + SSMS, Node.js backend, Tailwind frontend, VS Code, and GitHub for collaboration.

### 1. Required software

- Git
- Node.js and npm
- Visual Studio Code
- SQL Server and SQL Server Management Studio (SSMS)
- Live Server extension in VS Code
- Thunder Client extension in VS Code (recommended for API testing)

### 2. Recommended repository structure

- backend/ -> Express API, DB connection, routes, controllers, services
- frontend/ -> HTML, JS, Tailwind source files, compiled CSS output
- database/ -> SQL scripts needed to recreate the project database
- docs/ -> setup guide, API route notes, team instructions

### 3. Database files that should be in GitHub

A teammate should not need your live SQL Server instance. They should only need the SQL files stored in the repository.

| File                                                | Purpose                                                                                  |
|-----------------------------------------------------|------------------------------------------------------------------------------------------|
| database/schema.sql                                 | Creates the database structure: tables, keys, constraints, and relationships.            |
| database/sample-data.sql or database/full-setup.sql | Loads the main data needed for hotels, rooms, users, bookings, and related records.      |
| database/image-updates.sql                          | Adds or updates dbo.hotel_image.image so the homepage displays the correct hotel images. |

### 4. Database setup process in SSMS

1. Open SSMS and connect to the correct SQL Server instance.

2. Create the project database if it does not already exist, or select the existing one.
3. Run schema.sql first.
4. Run sample-data.sql next. If the team is using one combined export instead, run full-setup.sql instead of schema.sql plus sample-data.sql.
5. Run image-updates.sql last so the homepage image links are reproduced exactly.
6. Verify that hotel, room, hotel\_image, and other required tables contain data.

Example verification queries:

```
SELECT COUNT(*) AS hotel_count FROM dbo.hotel;
```

```
SELECT COUNT(*) AS room_count FROM dbo.room;
```

```
SELECT hotel_id, image FROM dbo.hotel_image ORDER BY hotel_id;
```

## 5. Backend setup

7. Open the backend folder in a terminal.
8. Run npm install to install dependencies.
9. Create a .env file using the project setup instructions.
10. Make sure the database connection settings point to the teammate's SQL Server instance and database name.
11. Start the backend with npm run dev.

Example backend .env fields:

```
DB_SERVER=YOUR_SERVER_NAME
```

```
DB_INSTANCE=SQLEXPRESS
```

```
DB_DATABASE=Hotel Booking System
```

```
PORT=3000
```

Example backend commands:

```
cd backend
```

```
npm install
```

```
npm run dev
```

## 6. Frontend setup

12. Open the frontend folder in a second terminal.
13. Install dependencies if needed.
14. Run the Tailwind watch/build command so output.css is generated and updated.
15. Open frontend/src/index.html with Live Server in VS Code.

Example frontend commands:

```
cd frontend
```

```
npm install
```

```
npm run dev:css
```

## 7. Startup order for teammates

16. Clone the GitHub repository.
17. Run the SQL scripts in SSMS.
18. Create the backend .env file.
19. Start the backend server.
20. Start the Tailwind CSS watch/build process.
21. Open the homepage with Live Server.

## 8. What a working setup should look like

- The backend responds at `http://localhost:3000/`.
- The hotels API returns JSON at `/api/hotels`.
- The homepage loads hotel cards in the Featured Hotels section.
- Only the selected featured hotels appear.
- Descriptions are cleaned for display, addresses are formatted, and images appear from `dbo.hotel_image.image`.

## 9. Recommended teammate test checklist

- Confirm the database contains hotels and hotel\_image rows.
- Confirm the backend starts without SQL connection errors.
- Open `/api/hotels` and verify it returns data.
- Open the homepage and verify images load.
- Confirm Tailwind styling is visible.
- Confirm the Featured Hotels section shows the intended number of hotels.

## 10. Troubleshooting

- **Cannot connect to database:** Check the SQL Server instance name, database name, Windows Authentication setup, and backend .env values.
- **Cannot GET or POST route errors:** Make sure the backend is running and the correct route is registered in `server.js`.
- **Homepage loads but no hotels appear:** Check `/api/hotels` first. If the API returns empty data, verify that `sample-data.sql` and `image-updates.sql` were run.
- **Images do not show:** Verify that `dbo.hotel_image.image` contains working URLs and that the hotels controller returns `hi.image AS image_url`.
- **Styles do not apply:** Make sure the Tailwind build/watch process is running and the page links to the compiled `output.css` file.

## 11. Recommended GitHub notes

- Commit SQL setup files to the repository.
- Do not commit secrets such as .env files.
- Keep a setup guide in docs/setup-guide.md.
- Ask teammates to work on feature branches instead of directly on main.

## 12. Summary

A teammate should be able to reproduce the project by cloning the repo, running the SQL scripts in order, adding backend environment variables, starting the backend, starting Tailwind, and opening the frontend. The key requirement is that the SQL files in GitHub must include both the database structure and the image link updates needed for the homepage.